

Pengujian Kapabilitas Deteksi *Anti Spyware Mode Stealth* dalam Pengumpulan Data Intelijen Siber

Proposal Tugas Akhir

Oleh

**Audra Zelvania Putri Harjanto
18222106**



**PROGRAM STUDI SISTEM DAN TEKNOLOGI INFORMASI
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
November 2025**

BAB IV

DESAIN KONSEP SOLUSI

Bab ini berisi penjelasan secara rinci mengenai rancangan sistem yang diusulkan dalam penelitian ini, yaitu desain modul *mission* dan *exfiltration* pada *spyware* dengan mode *stealth* yang digunakan untuk menguji kapabilitas deteksi *anti-spyware* dalam lingkungan terisolasi.

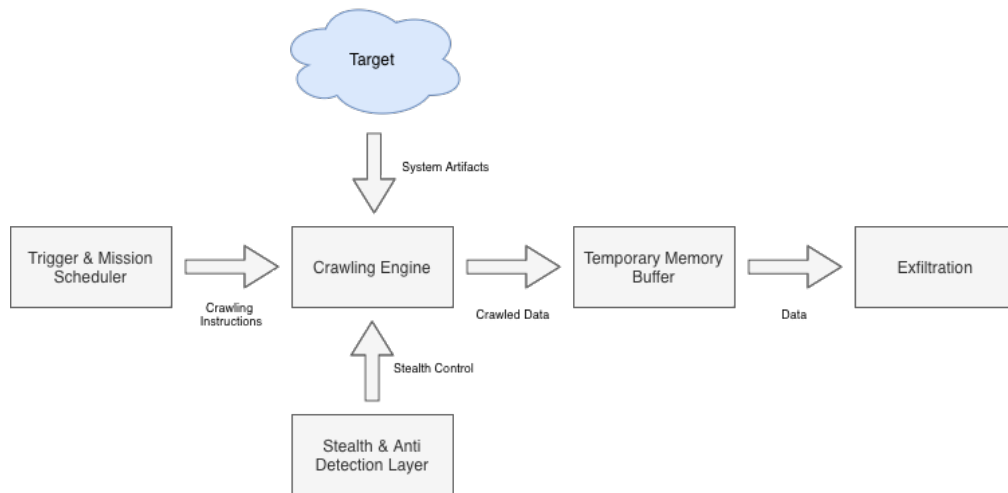
IV.1 Tahapan Desain

IV.1.1 Rancangan Modul *Mission*

Rancangan modul *Mission* disusun untuk mendefinisikan bagaimana proses pengambilan data (*data crawling*) dijalankan secara tersembunyi (*stealth*) pada sistem target. Modul ini berfungsi sebagai komponen utama yang menangani identifikasi artefak target, eksekusi proses *crawling* secara terjadwal, serta pengendalian tingkat aktivitas agar tidak menimbulkan pola perilaku mencurigakan. Seluruh mekanisme di dalamnya dirancang dengan pendekatan *fileless execution*, pengacakan interval, dan lapisan *anti detection* untuk memastikan bahwa proses berjalan tanpa memicu peringatan dari mekanisme keamanan. Bagian ini mencakup arsitektur modul, jenis data yang menjadi target *crawling*, lingkungan operasi, serta teknik-teknik *crawling stealth* yang digunakan dalam pengembangan sistem.

IV.1.1.1 Arsitektur Modul *Mission*

Modul *mission* berfungsi sebagai inti proses *crawling* data dalam sistem *spyware* mode *stealth*, dengan tujuan utama melakukan pengumpulan informasi secara diam-diam tanpa terdeteksi perangkat keamanan. Arsitektur modul ini mengutamakan teknik *fileless* dan *evasion* agar proses kerja tidak meninggalkan artefak digital di disk, serta meminimalisir kemungkinan identifikasi oleh *anti spyware*.



Gambar IV.1 Diagram Blok Arsitektur Modul *Mission*

Secara umum, arsitektur modul *Mission* yang akan dirancang terdiri dari empat komponen utama, yaitu:

1. *Trigger & Mission Scheduler*

Trigger & mission scheduler merupakan lapisan yang bertanggungjawab memulai eksekusi misi berdasarkan interval waktu, kondisi sistem, atau pemicu tertentu. *Scheduler* memastikan bahwa proses *crawling* tidak berjalan secara terus-menerus, melainkan dieksekusi dalam *batch* kecil dengan pola waktu yang acak. Pendekatan ini bertujuan untuk menghindari karakteristik eksekusi berulang dan berurutan, karena pola tersebut sering dianggap mencurigakan oleh sistem deteksi berbasis perilaku.

2. *Data Crawling Engine*

Komponen *crawling engine* merupakan inti dari proses pengambilan data pada sistem target. *Engine* ini menangani tugas enumerasi artefak, pembacaan *file* atau metadata, serta pengambilan informasi sensitif lainnya yang relevan. Proses *crawling* dirancang agar bersifat modular sehingga dapat menyesuaikan jenis artefak yang dituju. Seluruh proses berjalan dengan intensitas yang dikendalikan, menggunakan API bawaan sistem dan teknik khusus seperti *low-frequency access*, sehingga aktivitas *crawling* menyerupai perilaku rutin sistem operasi. Dengan begitu, *crawling engine* dapat mengakses data yang diperlukan tanpa menimbulkan lonjakan I/O atau pola akses abnormal yang dapat memicu alarm *anti spyware*.

3. *Stealth & Anti Detection Layer*

Komponen *stealth & anti detection layer* berfungsi untuk menjaga agar seluruh aktivitas modul *Mission* tetap tersembunyi dari mekanisme *anti spyware*.

Pada lapisan ini diimplementasikan *anti VM*, *anti sandbox*, dan *anti debugging* yang memungkinkan modul mendeteksi lingkungan yang tidak aman dan menunda proses sampai kondisi normal. Selain itu, lapisan ini mengontrol batasan intensitas *crawling* melalui pengaturan waktu jeda, pembatasan durasi eksekusi, serta pengacakan interval. Teknik *living-off-the-land* (LOTL) seperti pemanfaatan PowerShell, WMI, atau proses-proses internal sistem juga digunakan untuk menyamarkan jejak eksekusi.

4. *Stealth & Anti Detection Layer*

Temporary memory buffer merupakan ruang penyimpanan sementara yang menampung hasil *crawling* sebelum proses eksfiltrasi dilakukan. Sesuai dengan prinsip *memory-resident* pada *fileless architecture*, *buffer* ini diletakkan sepenuhnya di memori untuk menghindari pembuatan artefak fisik pada disk yang dapat menjadi bukti forensik. Data yang terkumpul disimpan dalam bentuk terstruktur dan dapat mengalami proses kompresi ringan agar lebih efisien. *Buffer* juga dilengkapi mekanisme penghapusan otomatis setelah sesi eksfiltrasi selesai, sehingga tidak meninggalkan residu yang dapat dianalisis. *Buffer* juga menjadi indikator kesiapan eksfiltrasi karena proses pengiriman data hanya dilakukan ketika kapasitas tertentu tercapai atau ketika misi menyelesaikan satu *batch crawling*.

Untuk mewujudkan arsitektur modul *Mission*, diperlukan pemilihan teknologi yang tepat agar setiap komponen dapat berfungsi sesuai karakteristiknya. Arsitektur tersebut memerlukan mekanisme eksekusi yang ringan, fleksibel, serta mampu berjalan sepenuhnya secara *fileless* tanpa meninggalkan jejak digital pada *disk*. Selain itu, modul *Mission* harus mendukung teknik *living-off-the-land*, implementasi *anti analysis*, serta pengelolaan data *in memory* agar aktivitas *crawling* dapat berlangsung secara *stealth* dan tidak memicu sistem deteksi *anti spyware*. Oleh karena itu, teknologi yang digunakan harus mampu mengakomodasi operasi *low level* pada sistem, menyediakan akses ke artefak sistem, serta sekaligus mendukung eksekusi yang tersamarkan. Pada Tabel IV.1 dibahas teknologi yang digunakan untuk mengimplementasikan setiap komponen modul *Mission* serta alasan pemilihannya.

Tabel IV.1 Teknologi yang Digunakan pada Setiap Komponen Modul *Mission*

Komponen Modul Mission	Teknologi yang Digunakan
Trigger & Mission Scheduler	• Python (time, random, asyncio)
Data Crawling Engine	• Python (os, pathlib, psutil, subprocess) • PowerShell • WMI • Win32 API
Stealth & Anti Detection Layer	• Win32 API • PowerShell • LOLBins
Temporary Memory Buffer	• Python (io.BytesIO, zlib)

IV.1.1.2 Target Data dan Lingkungan Operasi

Modul *Mission* dirancang untuk mengumpulkan data yang relevan dengan pemetaan aktivitas sistem dan pengguna, serta mendukung proses analitik pada tahap selanjutnya. Data yang dikumpulkan berfokus pada artefak yang dapat memberikan gambaran mengenai perilaku sistem tanpa mengambil konten sensitif secara langsung. Secara garis besar, terdapat X kelompok utama data yang menjadi target *crawling*.

Tabel IV.2 Target Data

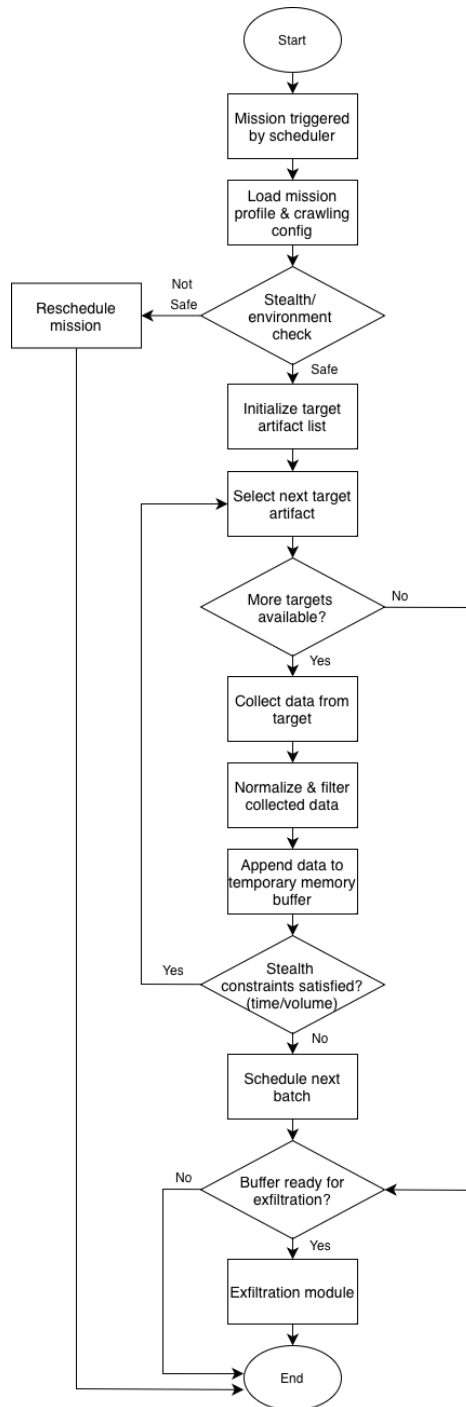
Kategori Data	Deskripsi	Atribut Utama	Contoh Nilai
Hasil <i>Keylogging</i>	Rekaman aktivitas penekanan tombol secara kronologis.	timestamp, key_code, process_name	—
Metadata Proses Sistem	Informasi konteks eksekusi proses pada saat aktivitas terjadi.	pid, process_name, cpu_usage, memory_usage, start_time, end_time	—
Artefak Sistem Tambahan	Artefak yang menggambarkan kondisi sistem saat <i>crawling</i> .	active_window, running_processes, focus_change	—

Seluruh proses *crawling* pada modul *Mission* dijalankan di dalam lingkungan terisolasi menggunakan *Virtual Machine (VM)* berbasis Windows 10/11 (64-bit). Penggunaan *VM* dipilih untuk memastikan bahwa pengujian berlangsung secara aman, tidak

memengaruhi sistem utama, serta memungkinkan proses *rollback* melalui *snapshot* apabila terjadi kesalahan selama eksperimen. Modul berjalan pada level pengguna (*user-level access*) tanpa hak administratif, sehingga mencerminkan kondisi bagaimana *spyware* berbasis *stealth* beroperasi. Selain itu, proses *crawling* hanya dilakukan apabila kondisi lingkungan dinilai aman untuk menjaga karakteristik *stealth* dari modul yang dirancang.

IV.1.1.3 Alur Kerja Modul *Mission*

Alur kerja modul *Mission* menggambarkan urutan proses yang dijalankan selama tahap pengumpulan data berlangsung, mulai dari pemicu eksekusi, pemeriksaan kondisi lingkungan, hingga penyimpanan data sementara dan persiapan eksfiltrasi. Secara keseluruhan, *workflow* dirancang agar proses *crawling* berjalan secara adaptif, terkontrol, dan tetap mempertahankan karakteristik *stealth* agar tidak terdeteksi oleh mekanisme pertahanan sistem. Diagram alur pada Gambar IV.2 memperlihatkan langkah-langkah utama yang dilakukan oleh modul ini.



Gambar IV.2 *Flowchart* Alur Kerja Modul *Mission*

Proses dimulai ketika *Mission Scheduler* memicu eksekusi modul sesuai interval waktu yang telah diacak untuk menghindari pola statis. Setelah modul mulai berjalan, konfigurasi misi dan profil *crawling* dimuat ke dalam memori untuk menentukan jenis data yang harus dikumpulkan pada *batch* eksekusi tersebut. Sebelum proses *crawling* dilakukan, modul menjalankan *stealth/environment check* untuk memas-

tikan bahwa kondisi perangkat aman. Pada tahap ini, sistem memeriksa indikasi keberadaan *sandbox*, *debugger*, atau aktivitas sistem yang tidak wajar. Jika lingkungan dianggap tidak aman, misi tidak dilanjutkan dan dijadwalkan ulang pada waktu berikutnya.

Apabila kondisi aman, modul *Mission* menginisialisasi daftar artefak target yang akan diproses, seperti metadata proses aktif atau *buffer keylogging* yang perlu dipindai. Modul kemudian memilih target berikutnya dari daftar, dan memeriksa apakah masih terdapat artefak yang harus dikumpulkan. Jika tidak ada target tersisa, *batch crawling* dianggap selesai dan proses dapat berlanjut ke tahap berikutnya.

Untuk setiap target yang tersedia, modul mengambil data melalui *crawling engine*. Proses ini mencakup pembacaan metadata sistem maupun pengambilan *buffer* hasil penekanan tombol. Data mentah yang diperoleh kemudian dinormalisasi, difilter, dan disesuaikan dengan format penyimpanan sementara agar tetap efisien dan konsisten. Setelah diproses, data ditambahkan ke dalam *temporary memory buffer* yang berada sepenuhnya di memori untuk menjaga sifat *fileless*.

Selanjutnya, modul melakukan evaluasi terhadap batasan *stealth*, seperti durasi eksekusi maksimum atau volume data per *batch*. Jika batasan belum terpenuhi, modul menjadwalkan *batch crawling* berikutnya dan kembali ke proses pemilihan target. Jika batasan terpenuhi, modul bergerak ke tahap selanjutnya, yaitu memeriksa apakah *buffer* siap untuk dieksfiltrasi. Pemeriksaan ini dapat berbasis ukuran *buffer*, jumlah entri yang terkumpul, atau kondisi khusus lainnya.

Jika *buffer* belum memenuhi kriteria eksfiltrasi, modul kembali ke proses *crawling* pada *batch* berikutnya. Namun, apabila *buffer* telah mencapai kondisi siap kirim, modul *Mission* mengaktifkan komponen eksfiltrasi dan menyerahkan data untuk dikirim ke modul berikutnya. Setelah proses eksfiltrasi selesai, eksekusi modul *Mission* pada *batch* tersebut diakhiri.

Dengan alur kerja ini, modul *Mission* dapat melakukan pengumpulan data secara bertahap, efisien, dan tetap mempertahankan sifat *stealth*. Pendekatan berbasis *batch*, validasi lingkungan, dan kontrol frekuensi eksekusi memastikan proses berjalan tanpa memicu sistem deteksi maupun meninggalkan artefak yang dapat dianalisis.

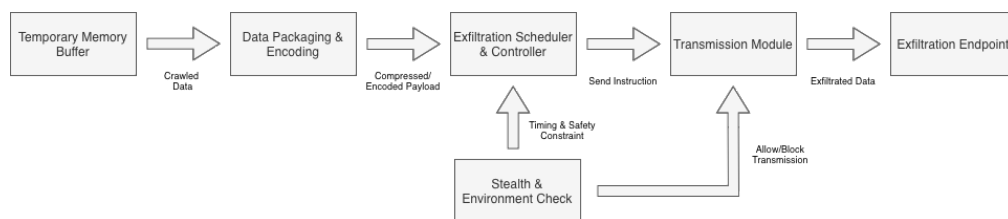
IV.1.2 Rancangan Modul *Exfiltration*

Rancangan modul *Exfiltration* disusun untuk mendefinisikan bagaimana data hasil *crawling* diproses, dikemas, dan dikirimkan ke *endpoint* secara aman serta tetap

mempertahankan karakteristik *stealth*. Modul ini berfungsi mengelola penjadwalan pengiriman, menyiapkan *payload* yang telah terstruktur, serta memastikan proses transmisi berlangsung tanpa memicu deteksi dari sistem keamanan. Mekanisme di dalamnya mencakup proses *packaging*, *encoding*, *compression*, pemilihan waktu dan metode pengiriman, serta kontrol lingkungan sebelum data dikirim. Bagian ini membahas arsitektur modul eksfiltrasi, jenis *output* yang dihasilkan, serta alur mekanisme pengiriman data dari *buffer* hingga diterima oleh *endpoint* analitik.

IV.1.2.1 Arsitektur Modul *Exfiltration*

Modul *Exfiltration* bertanggungjawab untuk mengirimkan data hasil *crawling* dari modul *Mission* ke penerima dalam bentuk yang terstruktur, aman, dan tetap mempertahankan sifat *stealth*. Modul *Exfiltration* berperan sebagai jembatan antara lingkungan eksekusi *spyware* dan *big data analytic*. Arsitektur modul ini dirancang agar dapat bekerja secara *fileless*, meminimalkan jejak pada *disk*, serta menggunakan pola pengiriman *low and slow* sehingga tidak memicu deteksi anomali jaringan.



Gambar IV.3 Diagram Blok Arsitektur Modul *Exfiltration*

Secara garis besar, arsitektur modul *Exfiltration* terdiri dari beberapa komponen utama, yaitu:

1. *Temporary Memory Buffer*

Temporary memory buffer berfungsi sebagai titik masuk (*entry point*) bagi modul *Exfiltration*. *Buffer* ini berisi kumpulan data hasil *crawling* modul *Mission* yang disimpan sepenuhnya di memori. Dengan menempatkan *buffer* di RAM, arsitektur ini menghindari pembuatan *file log* permanen pada *disk* sehingga konsisten dengan pendekatan *fileless*. Modul *Exfiltration* tidak berinteraksi langsung dengan modul *Mission*, melainkan hanya membaca data yang sudah siap dari *buffer* tersebut.

2. *Data Packaging & Encoding*

Data packaging & encoding bertugas mengubah data mentah dalam *buffer* menjadi *payload* yang siap dikirim. Pada tahap ini dilakukan proses normalisasi, penggabungan, kompresi, serta encoding data. Tujuannya adalah meng-

urangi ukuran *payload*, menyamakan format antar *batch*, dan mempermudah proses pengiriman melalui protokol jaringan yang digunakan. Dengan demikian, data yang dikirim ke *endpoint* bersifat ringkas, konsisten, dan tidak mudah dikenali sebagai *traffic* yang tidak biasa.

3. *Exfiltration Scheduler & Controller*

Komponen *exfiltration scheduler & controller* berperan sebagai pengatur waktu dan ritme pengiriman. Modul ini menentukan kapan proses eksfiltrasi dijalankan, seberapa besar ukuran *batch* yang dikirim, serta mengatur interval pengiriman agar tidak menimbulkan lonjakan lalu lintas jaringan. Pendekatan *low frequency transmission* dan *randomized interval* diterapkan juga diterapkan sehingga pola pengiriman tidak tampak statis atau berulang. *Scheduler* juga mengelola status antrian pengiriman, misalnya menandai *payload* yang telah berhasil dikirim maupun yang perlu diulang.

4. *Stealth & Environment Check*

Komponen *stealth & environment check* yang memonitor kondisi lingkungan sebelum dan selama proses eksfiltrasi. Komponen ini melakukan pengecekan terhadap indikator tertentu, seperti beban CPU, ketersediaan jaringan, maupun adanya tanda-tanda lingkungan analisis (*sandbox*, *debugger*, atau *monitoring* intensif). Hasil pengecekan ini digunakan sebagai *constraint* bagi *exfiltration scheduler & controller* dan *transmission module*. Jika kondisi tidak memenuhi kriteria *stealth*, modul dapat menunda pengiriman, memperpanjang interval, atau membatalkan proses eksfiltrasi untuk *batch* tersebut.

5. *Transmission Module*

Komponen *transmission module* merupakan bagian yang melakukan pengiriman data secara nyata ke pihak penerima. Modul ini mengolah *payload* yang sudah dikemas dan apabila sudah diizinkan oleh *stealth layer* mengirimkannya melalui saluran komunikasi yang telah ditentukan. *Transmission module* didesain agar menyerupai trafik aplikasi biasa, sehingga tidak mudah dibedakan dari aktivitas jaringan normal oleh mekanisme pemantauan.

6. *Exfiltration Endpoint*

Exfiltration endpoint merupakan komponen di sisi penerima yang menerima *payload* dari modul *exfiltration*. *Endpoint* ini berupa server penerima di lingkungan terkontrol yang kemudian akan meneruskan data tersebut ke sistem penyimpanan yang akan diolah dalam *big data analytic*.

IV.1.2.2 Target Output *Exfiltration*

Output eksfiltrasi merupakan data akhir yang telah dikumpulkan oleh modul *Mission* dan diproses sebelum dikirimkan ke *endpoint* tujuan. Secara umum, output yang dieksfiltrasi terdiri dari tiga kategori utama yang disajikan pada Tabel IV.3.

Tabel IV.3 Target *Output Exfiltration*

Kategori Output	Deskripsi	Atribut Utama	Contoh Nilai
Hasil <i>Keylogging</i>	Rekaman aktivitas penekanan tombol secara kronologis.	timestamp, key_code, process_name	—
Metadata Proses Sistem	Informasi konteks eksekusi proses pada saat aktivitas terjadi.	pid, process_name, cpu_usage, memory_usage, start_time, end_time	—
Artefak Sistem Tambahan	Artefak yang menggambarkan kondisi sistem saat <i>crawling</i> .	active_window, running_processes, focus_change	—

Sebelum dikirimkan, seluruh data dikemas dalam struktur *payload* yang sudah dikompresi dan di-*encode*. *Payload* dapat berupa JSON terstruktur sebelum diberikan proses kompresi. Struktur *payload* dipaparkan pada Tabel IV.4.

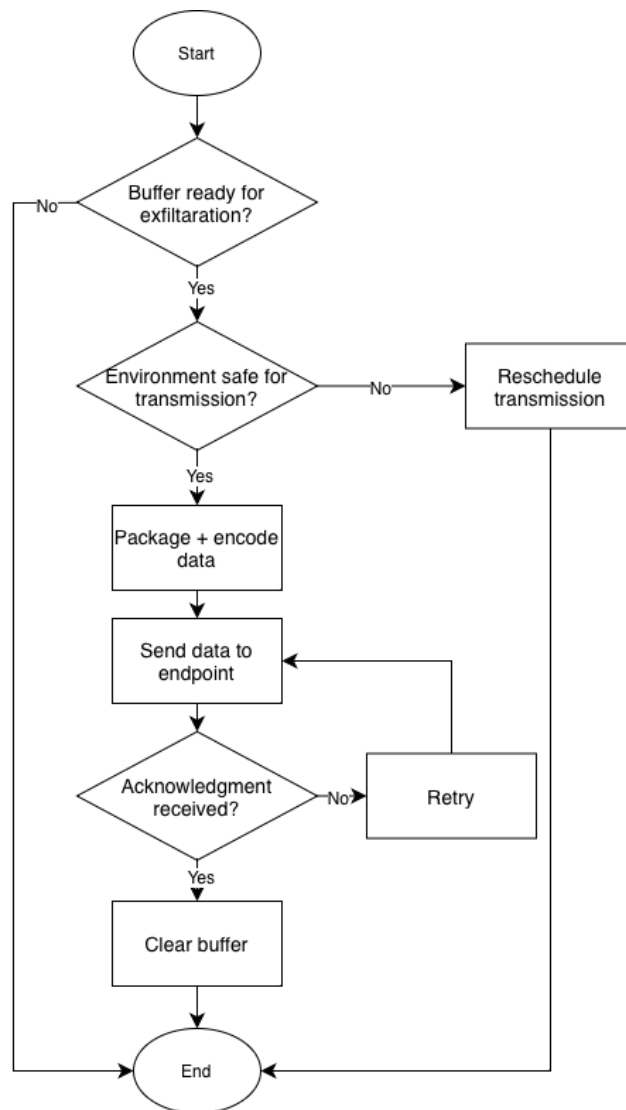
Tabel IV.4 Struktur *Payload*

<i>Field</i>	Isi/Struktur
keylogs	List objek <i>keylogging</i>
metadata	List metadata proses
system_artifacts	Data artefak sistem
session_id	ID unik <i>batch crawling</i>
generated_at	<i>Timestamp</i> waktu <i>payload</i> dibuat

IV.1.2.3 Mekanisme Pengiriman Data

Mekanisme pengiriman data pada modul *Exfiltration* dirancang untuk memastikan proses transfer berjalan secara aman, terkontrol, dan tidak menimbulkan pola komunikasi yang mudah dikenali oleh sistem pertahanan. Alur kerja ini digambarkan melalui diagram alur pada Gambar IV.4, dimulai dari pemeriksaan kesiapan *buffer*,

penjadwalan waktu pengiriman, hingga proses pembersihan *buffer* setelah data berhasil dikirimkan.



Gambar IV.4 *Flowchart* Mekanisme Pengiriman Data pada Modul *Exfiltration*

Proses dimulai dengan memeriksa apakah *temporary memory buffer* telah mencapai kondisi siap untuk dikirim, baik berdasarkan ukuran muatan (*payload size threshold*) maupun batas waktu (*time threshold*). Jika *buffer* belum siap, *batch* dihentikan dan modul menunggu siklus berikutnya yang dipicu kembali oleh *scheduler*.

Apabila *buffer* berada dalam kondisi siap, modul akan melakukan evaluasi lingkungan. Tahap ini memastikan bahwa sistem target berada dalam keadaan aman untuk melakukan komunikasi keluar. Jika kondisi dinilai tidak aman, modul tidak memaksakan pengiriman dan akan menjadwalkan ulang transmisi pada interval ber-

ikutnya.

Jika lingkungan aman, sistem memasuki tahap *data packaging & encoding*. Pada tahap ini, data hasil *crawling* yang tersimpan di memori dikompresi dan di-*encode* menggunakan teknik ringan agar ukuran paket lebih kecil dan pola data lebih sulit dianalisis. *Payload* yang sudah dipaketkan kemudian dikirimkan ke *endpoint* tujuan melalui modul transmisi.

Setelah pengiriman dilakukan, modul akan menunggu respons berupa *acknowledgment* dari *endpoint*. Jika *acknowledgment* tidak diterima, modul akan mencoba melakukan *retry* pengiriman sejumlah batas maksimum yang telah ditentukan. Proses *retry* tetap berada dalam *batch* yang sama. Jika seluruh percobaan gagal, modul akan mengakhiri *batch* tanpa menghapus *buffer* sehingga data tetap aman dan akan dicoba kembali pada siklus berikutnya.

Apabila *acknowledgment* diterima, *buffer* dianggap telah berhasil dieksfiltrasi. Modul kemudian melakukan *buffer clearing* untuk menghapus seluruh data yang telah dikirim dari memori, sesuai dengan prinsip *memory resident operation* pada arsitektur *fileless*. Tahap ini sekaligus menandai berakhirnya satu siklus eksfiltrasi, dan modul kembali menunggu pemicu berikutnya dari *scheduler*.

IV.1.2.4 Rancangan Lingkungan Pengujian

Lingkungan pengujian dirancang agar proses eksekusi modul *Mission* dan *Exfiltration* dapat dilakukan secara aman dan terkontrol. Seluruh percobaan dilakukan pada lingkungan Windows yang dikonfigurasi secara terisolasi sehingga aktivitas *crawling* dan eksfiltrasi tidak berdampak pada sistem di luar area pengujian. Lingkungan uji yang digunakan disajikan pada Tabel IV.5.

Tabel IV.5 Konfigurasi Lingkungan Pengujian

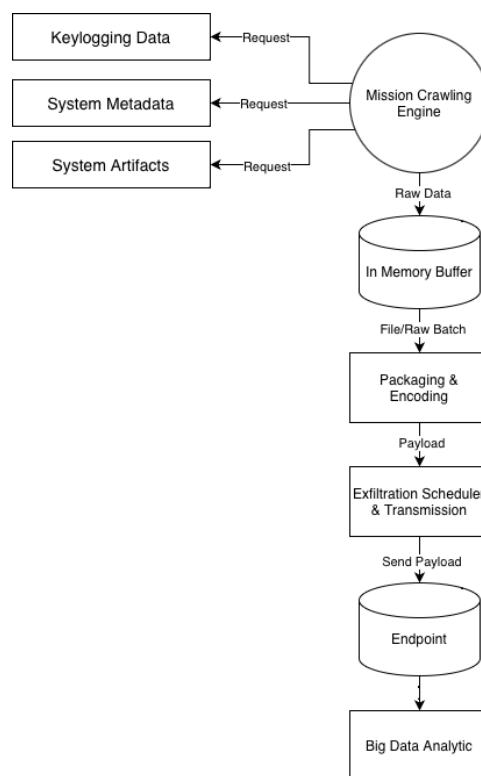
Komponen	Detail Konfigurasi
Sistem Operasi	Windows 10/11
Bahasa Pemrograman	Python 3.x, PowerShell
Library Utama	psutil, pathlib, requests, Win32API
Endpoint Eksfiltrasi	REST API (FastAPI)

Lingkungan pengujian pada penelitian ini dibangun menggunakan sistem operasi Windows 10/11 yang dikonfigurasi secara terisolasi untuk memastikan bahwa pro-

ses *crawling* dan *eksfiltrasi* tidak berdampak pada sistem di luar area eksperimen. Seluruh modul dijalankan menggunakan Python 3.x dan PowerShell sebagai bahasa pemrograman utama, dengan dukungan beberapa *library* seperti *psutil*, *pathlib*, *requests*, dan Win32API untuk mengakses artefak sistem serta mengelola proses *crawling*. Proses *exfiltration* diarahkan menuju sebuah *endpoint* lokal berbasis *REST API* (FastAPI) yang berfungsi sebagai server penerima *payload* hasil *crawling*. Konfigurasi ini memastikan bahwa seluruh aktivitas uji dapat dilakukan secara aman, terkontrol, dan sesuai dengan prinsip penelitian *spyware mode stealth*.

IV.2 Hasil Desain

Bagian ini merupakan hasil desain dari arsitektur sistem yang mencakup keseluruhan proses *mission crawling* hingga *data exfiltration*, yang menjadi fokus utama pada penelitian ini. Desain ini disusun untuk menunjukkan bagaimana modul-modul yang dibangun berinteraksi, bagaimana data bergerak dari sumber hingga tujuan akhir, serta bagaimana prinsip *stealth* dipertahankan sepanjang alur eksekusi.



Gambar IV.5 Diagram Hasil Desain Arsitektur Sistem

Pada desain ini, sistem memulai proses dengan tiga kelompok data utama, yaitu *keylogging data*, *system metadata*, dan *system artifacts*. Ketiganya diakses oleh *Mis-*

sion Crawling Engine secara terjadwal melalui mekanisme *request based crawling*. *Engine* kemudian menghasilkan *raw data* yang disimpan di *In Memory Buffer*, yaitu ruang penyimpanan sementara yang bersifat *memory resident* untuk menghindari terbentuknya jejak digital pada *disk*.

Selanjutnya, data dari *buffer* diproses dalam tahap *Packaging & Encoding*, di mana data dikemas, di-*encode*, dan dikompresi menjadi sebuah *payload* terstruktur. *Payload* ini kemudian dikirim melalui *Exfiltration Scheduler & Transmission Module*, yang bekerja dengan memperhatikan batasan *stealth* seperti pengacakan waktu pengiriman, pemilihan metode transmisi yang rendah anomali, serta mekanisme *retry* bila konfirmasi tidak diterima. Data yang berhasil dieksfiltrasi akan tersimpan dalam *Endpoint*, dan selanjutnya menjadi input bagi *Big Data Analytic* untuk proses analisis lebih lanjut.